



Building RESTful APIs for Your Pet Store: A Developer's Guide

Section 1: Learn

What is the Pet Store API Project?

This project demonstrates how to build a RESTful API for managing pets and users in a store. It includes full CRUD functionality, validation, layered architecture, and integration with a MySQL database.

Why Build This API?

- **Real-World Practice:** Understand how APIs are structured in modern backend systems.
- **Clean Architecture:** Follow separation of concerns via controllers, services, repositories.
- **Validation and DTO Usage:** Safeguard data input and streamline communication between layers.

API Architecture

- **Controllers:** Handle incoming HTTP requests and delegate processing.
- **Services:** Contain business logic and validation.
- **Repositories:** Manage data persistence.
- **Entities:** Map Java classes to database tables via JPA.

Interesting Anecdote

This project mirrors the structure found in real-world backend systems for e-commerce or product management platforms. Learning this gives you the confidence to build any REST API service with Spring Boot.



Section 2: Practice

Pet Entity API

PetController.java

```
@RestController
@RequestMapping("/pets")
public class PetController {

    @Autowired
    private PetService petService;

    @PostMapping
    public pet addPet(@RequestBody pet pet) {
        return petService.addPet(pet);
    }

    @GetMapping
    public List<pet> getAllPets() {
        return petService.getAllPets();
    }

    @GetMapping("/{id}")
    public pet getPetById(@PathVariable Integer id) {
        return petService.getPetById(id);
    }
}
```



```
@PutMapping("/{id}")
public pet updatePet(@PathVariable Integer id, @RequestBody pet pet) {
    return petService.updatePet(id, pet);
}

@DeleteMapping("/{id}")
public String deletePet(@PathVariable Integer id) {
    petService.deletePet(id);
    return "Pet deleted with ID: " + id;
}
}
```

PetService.java

```
@Service
public class PetService {
    @Autowired
    private PetRepository petRepository;

    public pet addPet(pet pet) {
        return petRepository.save(pet);
    }

    public List<pet> getAllPets() {
        return petRepository.findAll();
    }
}
```



```
public pet getPetById(Integer id) {  
    return petRepository.findById(id).orElse(null);  
}  
  
public pet updatePet(Integer id, pet newPetData) {  
    Optional<pet> optionalPet = petRepository.findById(id);  
    if (optionalPet.isPresent()) {  
        pet existingPet = optionalPet.get();  
        existingPet.setName(newPetData.getName());  
        existingPet.setBreed(newPetData.getBreed());  
        existingPet.setAge(newPetData.getAge());  
        existingPet.setPrice(newPetData.getPrice());  
        existingPet.setQuantity(newPetData.getQuantity());  
        existingPet.setImagePath(newPetData.getImagePath());  
        return petRepository.save(existingPet);  
    }  
    return null;  
}  
  
public void deletePet(Integer id) {  
    petRepository.deleteById(id);  
}  
}
```



User Entity API

UserController.java

```
@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @PostMapping
    public User createUser(@Valid @RequestBody userdto dto) {
        return userService.createUser(dto);
    }

    @GetMapping
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }

    @GetMapping("/{id}")
    public User getUserById(@PathVariable Long id) {
        return userService.getUserById(id);
    }

    @PutMapping("/{id}")
```



```
public User updateUser(@PathVariable Long id, @Valid @RequestBody
userdto dto) {
    return userService.updateUser(id, dto);
}

@DeleteMapping("/{id}")
public String deleteUser(@PathVariable Long id) {
    userService.deleteUser(id);
    return "User with ID " + id + " deleted successfully.";
}
}
```

UserService.java

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public User createUser(userdto dto) {
        User user = new User();
        user.setUsername(dto.getName());
        user.setEmail(dto.getEmail());
        user.setPassword(dto.getPassword());
        return userRepository.save(user);
    }
}
```



```
public List<User> getAllUsers() {  
    return userRepository.findAll();  
}
```

```
public User getUserById(Long id) {  
    return userRepository.findById(id).orElse(null);  
}
```

```
public User updateUser(Long id, userdto dto) {  
    Optional<User> optionalUser = userRepository.findById(id);  
    if (optionalUser.isPresent()) {  
        User user = optionalUser.get();  
        user.setUsername(dto.getName());  
        user.setEmail(dto.getEmail());  
        user.setPassword(dto.getPassword());  
        return userRepository.save(user);  
    }  
    return null;  
}
```

```
public void deleteUser(Long id) {  
    userRepository.deleteById(id);  
}
```

```
}
```



Testing with Postman

1. Create User (POST)

- URL: <http://localhost:8080/users>
- Method: POST
- Body: raw JSON

2. Get All Users (GET)

- URL: <http://localhost:8080/users>

3. Get User by ID (GET)

- URL: <http://localhost:8080/users/{id}>

4. Update User (PUT)

- URL: <http://localhost:8080/users/{id}>

5. Delete User (DELETE)

- URL: <http://localhost:8080/users/{id}>

Section 3: Know More

FAQs

Q1: What is the purpose of separating DTO and Entity?

- **Answer:** DTOs are used for transferring data and apply validation, while Entities are used for persistence and database structure.

Q2: How does Spring Boot convert JSON to Java objects?

- **Answer:** Spring Boot uses the Jackson library internally to map JSON input to Java classes using [@RequestBody](#).

Q3: What if an ID doesn't exist during update/delete?

- **Answer:** The service layer checks using [Optional.isPresent\(\)](#) to avoid null pointer exceptions.

Q4: How is Postman used in this project?



- **Answer:** Postman is used to test each endpoint manually by sending HTTP requests and verifying the response.

Q5: Can I use Lombok in this project?

- **Answer:** Yes. Lombok can reduce boilerplate code for getters, setters, and constructors using annotations like `@Data`.